

Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

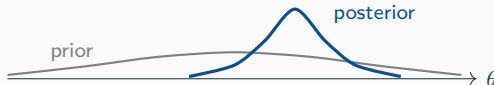
Background: neural Bayes estimators

Bayesian inference

- Fix a model: parameters θ with prior $p(\theta)$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} | \theta)$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior**:

$$p(\theta | \mathbf{y}) \propto p(\mathbf{y} | \theta) p(\theta).$$

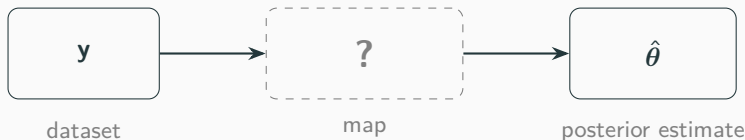
- The posterior is our full, updated belief about θ .



Data pulls the wide prior toward a sharper, updated belief.

Point estimation

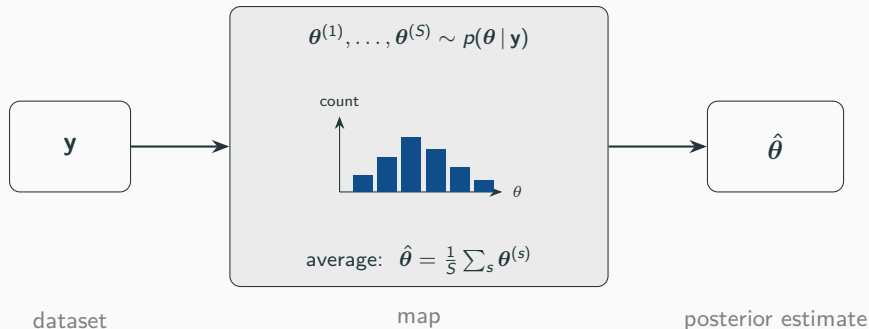
- Often we do not want the whole posterior — just a single **point estimate** $\hat{\theta}$ of θ .
- The usual choice is the **posterior mean** $\hat{\theta} = \mathbb{E}[\theta | \mathbf{y}]$ (best under squared error — more soon).
- But it is **rarely available in closed form** — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\theta}$:



The next two slides fill this box in two different ways.

Filling the box (1): Markov chain Monte Carlo

The box *is* a sampler: draw from the posterior, histogram them, average.

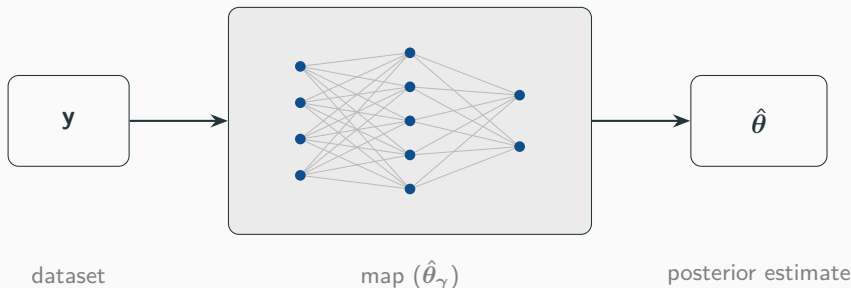


- Accurate and general — but must be **re-run from scratch for every new y** , and can be slow.

Filling the box (2): a neural network

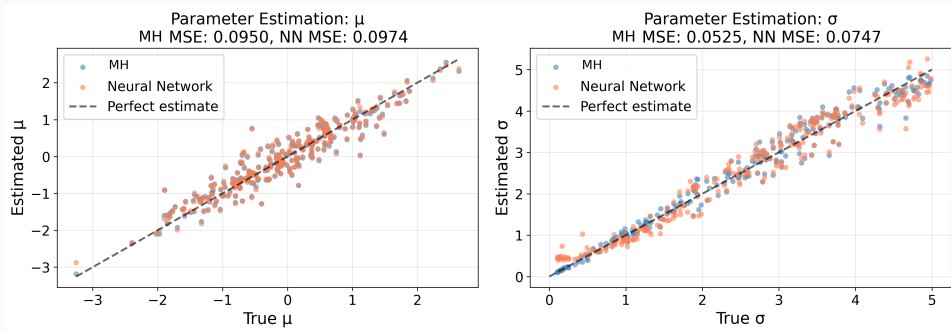
Make the box a **neural network** $\hat{\theta}_{\gamma}(\mathbf{y})$, trained *once* — a *neural Bayes estimator*.

- Training data is free: **simulate** pairs $(\theta, \mathbf{y})$ from the model, and adjust γ so $\hat{\theta}_{\gamma}(\mathbf{y}) \approx \theta$.
- Then each new dataset is a single **forward pass** — the cost is **amortised**.



Does it work?

A simple test: estimate the mean μ and scale σ of a normal model — network vs. MCMC, on held-out data.



The network (orange) matches MCMC (blue): it has **learned the posterior-mean map**. *Why* it does is the next slide.

Our contribution: variance reduction

What is the network trained to do?

It minimises the **Bayes risk** — expected loss over the model:

$$L(\gamma) = \mathbb{E}_{\theta, \mathbf{y}} [\ell(\theta, \hat{\theta}_{\gamma}(\mathbf{y}))].$$

- Under squared error, the minimiser is exactly the **posterior mean**:

$$\arg \min_f \mathbb{E} [\|\theta - f(\mathbf{y})\|^2] = \mathbb{E}[\theta | \mathbf{y}].$$

That is why a well-trained network approximates $\mathbb{E}[\theta | \mathbf{y}]$ — it explains the last slide.

- Change the loss, change the summary: absolute error $\rightarrow$ median, quantile loss $\rightarrow$ quantiles.

Pointwise in $\mathbf{y}$, the best constant c minimising $\mathbb{E}[\|\theta - c\|^2 | \mathbf{y}]$ is $c = \mathbb{E}[\theta | \mathbf{y}]$.

The observation this thesis starts from

We never have the true risk — we estimate it by **Monte Carlo** over simulated draws:

$$L(\gamma) \approx \frac{1}{N} \sum_{i=1}^N \|\theta_i - \hat{\theta}_{\gamma}(\mathbf{y}_i)\|^2, \quad (\theta_i, \mathbf{y}_i) \text{ simulated.}$$

- So the loss — and the **gradient** that drives training — is a **random quantity with variance**.
- That variance comes from *how we sample the loss*, not from the inference problem.
- **So we can reduce it**, with the classical variance-reduction tools of Monte Carlo.

Idea 1: Rao–Blackwellisation

A classical trick for killing Monte Carlo noise.

- Estimating an average by sampling? If part of it can be computed **exactly**, do that — replace the sampled piece by its exact **conditional mean**.
- Conditioning-and-averaging can only *remove* variance, never add it.

noisy draws $\longrightarrow$ replace one block by its exact mean $\longrightarrow$ tighter estimate

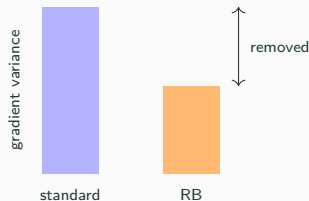
Named after the Rao–Blackwell theorem.

Rao–Blackwellising the loss — and why it must help

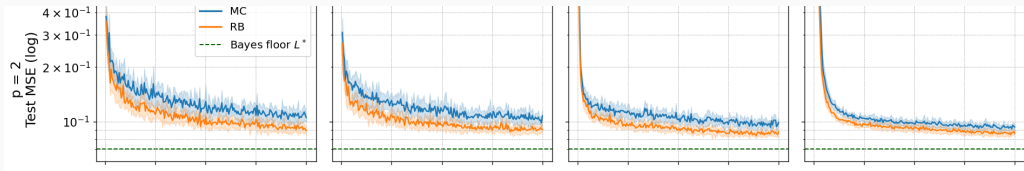
Split $\theta = (\tau, \beta)$ with β conditionally conjugate; integrate β out by replacing its sampled target with $\mathbb{E}[\beta \mid \tau, \mathbf{y}]$. For the gradient g ,

$$\text{Var}(g) = \underbrace{\text{Var}(\mathbb{E}[g \mid \tau, \mathbf{y}])}_{\text{Var}(g_{\text{RB}})} + \underbrace{\mathbb{E}[\text{Var}(g \mid \tau, \mathbf{y})]}_{\succeq 0}.$$

- Law of total variance $\Rightarrow \text{Var}(g_{\text{RB}}) \preceq \text{Var}(g)$.
- **Never increases gradient variance** — any model, any architecture.
- Linear net: the reduction reaches the fitted weights too.



Does it pay off? Bayesian linear regression



MC (blue) vs RB (orange) vs Bayes floor L^* (green) — across batch sizes 4, 16, 64, 256.

lower loss in
every grid cell

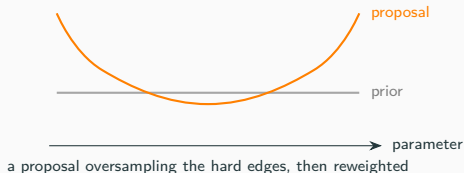
closes $\sim$ **half** (52%)
of the gap to L^*

matches the standard NBE
at $\frac{1}{4}$ **the batch size**

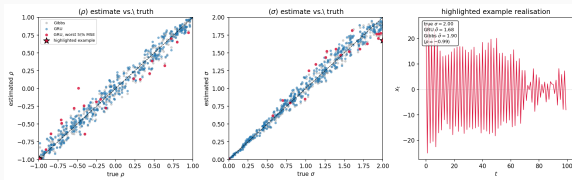
Idea 2: importance sampling

A more general lever — **no conjugacy required**.

- Simulate from a **proposal** that oversamples the informative regions, then **reweight** back to the original risk (unbiased for any proposal).
- But **no guarantee**: bad weights can *increase* variance and shrink the effective sample size.

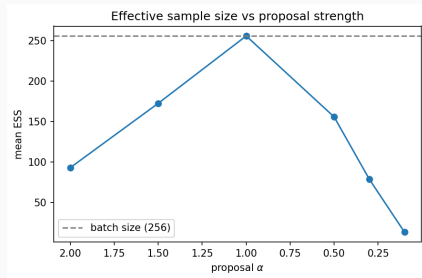


Importance sampling: the result



On an AR(1) model, the GRU already tracks the Gibbs (Bayes) reference across the prior — **little error left to recover.**

General, yes — but here reweighting found no error worth recovering. **No guarantee, and it did not help.**



Push the proposal hard and the effective sample size **collapses** — only adding noise.

Conclusion

Conclusion

Training a neural Bayes estimator is itself Monte Carlo — so its training variance can be reduced.

- **Rao–Blackwellisation** (needs a conjugate block): a *provable* reduction — closes $\sim$ half the gap to the Bayes floor and matches the standard estimator at $\frac{1}{4}$ the batch size.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Conjugacy — the classical trick behind Gibbs sampling — is just as useful for *training* the estimators meant to replace it.

Thank you